MILESTONE 1: HEALTH

Section A

A1. ISSUES OBSERVED:

1.Mixing every entities in one table (i.e separate table need to be created for patient,doctor,medicine,appointment,billing).

2.we know that there can be only one primary key in table and given schema can give result to duplicate data.

3.also facing relationship connection bcoz if we do different table and there each table if we want to connect other table according to it's relation then foreign key can be used to match relation .(but here fails to satisfies)

4.While deleting one record it can delete all useful data or information(ie.while deleting patient details it will delete all medicine and doctor information)

5.voilates 1NF(repeatation,wasting storage)

A2. Normalise the schema

1. 1NF:

```
CREATE TABLE  PatientRecords (
   patient_id INT,
   appointment_date DATE,
   patient_name VARCHAR(10),
   patient_phone VARCHAR(12),
   doctor_name VARCHAR(10),
   doctor_specialization VARCHAR(10),
   diagnosis VARCHAR(100),
   treatment VARCHAR(100),
   prescription_details VARCHAR(255),
   bill_amount INT,
   payment_status VARCHAR(20),

   PRIMARY KEY (patient_id, appointment_date)
);

CREATE TABLE Medicines (
   patient_id INT,
   appointment_date DATE,
   medicine_name VARCHAR(20),
   medicine_price INT,
   PRIMARY KEY (patient_id, appointment_date, medicine_name),
```

```
   FOREIGN KEY (patient_id, appointment_date)
 REFERENCES PatientRecords(patient_id, appointment_date)
);
```

This can be done for satisfying 1NF, for a different patient there can be medicine information different so by removing repeating groups (medicine details).

2. 2NF (partial dependency):
   Patient name ,patient phone depends on patient id but doctor name,doctor specialisation is not dependent.

   here we still break the tables to satisfy 2NF.

```
CREATE TABLE Patients (
   patient_id INT PRIMARY KEY,
   patient_name VARCHAR(10),
   patient_phone VARCHAR(15)
);

CREATE TABLE  Doctors (
   doctor_id INT PRIMARY KEY,
   doctor_name VARCHAR(10),
   doctor_specialization VARCHAR(10)
);

CREATE TABLE Appointments (
   appointment_id INT PRIMARY KEY,
   patient_id INT,
   doctor_id INT,
   appointment_date DATE,
   diagnosis VARCHAR(100),
   treatment VARCHAR(10),

   FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),
   FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id)
);

CREATE TABLE  Medicines (
   appointment_id INT,
```

```
  medicine_name VARCHAR(20),
  medicine_price  INT,
  PRIMARY KEY (appointment_id, medicine_name),
  FOREIGN KEY (appointment_id) REFERENCES Appointments(appointment_id)
);

CREATE TABLE Billing (
  appointment_id INT PRIMARY KEY,
  bill_amount INT,
  payment_status VARCHAR(20),

  FOREIGN KEY (appointment_id) REFERENCES Appointments(appointment_id)
);
```

3NF:

```
CREATE TABLE Patients (
  patient_id INT PRIMARY KEY,
  patient_name VARCHAR(50),
  patient_phone VARCHAR(15)
);

CREATE TABLE Doctors (
  doctor_id INT PRIMARY KEY,
  doctor_name VARCHAR(10),
  doctor_specialization VARCHAR(10)
);

CREATE TABLE Appointments (
  appointment_id INT PRIMARY KEY,
  patient_id INT,
  doctor_id INT,
  appointment_date DATE,
  diagnosis VARCHAR(100),
  treatment VARCHAR(10),

  FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),
  FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id)
);
CREATE TABLE Medicines (
  medicine_id INT PRIMARY KEY,
```

```
  medicine_name VARCHAR(10),
  medicine_price INT
);

CREATE TABLE  Prescriptions (
  prescription_id INT PRIMARY KEY,
  appointment_id INT,
  FOREIGN KEY (appointment_id) REFERENCES Appointments(appointment_id)
);

CREATE TABLE Prescription_Details (
  prescription_id INT,
  medicine_id INT,
  PRIMARY KEY (prescription_id, medicine_id),
  FOREIGN KEY (prescription_id) REFERENCES Prescriptions(prescription_id),
  FOREIGN KEY (medicine_id) REFERENCES Medicines(medicine_id)
);

CREATE TABLE Billing (
  bill_id INT PRIMARY KEY,
  appointment_id INT,
  bill_amount INT,
  payment_status VARCHAR(20),
  FOREIGN KEY (appointment_id) REFERENCES Appointments(appointment_id)
);
```

A3. Assumptions

1.patient and appointment relation(a patient can have multiple appointments)
2.doctor and appointment relation(doctor can have to attend multiple appointments)
3.prescription and medicine relation(each prescription can have multiple medicines)
4.many to many relation
5 .same medicine can be used to manhy prescription.

Section B :

B1. Requirement Clarification

patient have **multiple phone numbers**?
patient email required?
doctor have **multiple specializations**?
one appointment -> **multiple doctors**?
Appointments be rescheduled or canceled?
Payment mode?

B2. Design
Design the database schema.
Your submission must include:

- Tables
- Attributes
- Primary Keys
- Foreign Keys

The design should be structured and complete.
B3. Normalization
Ensure the schema is normalized.

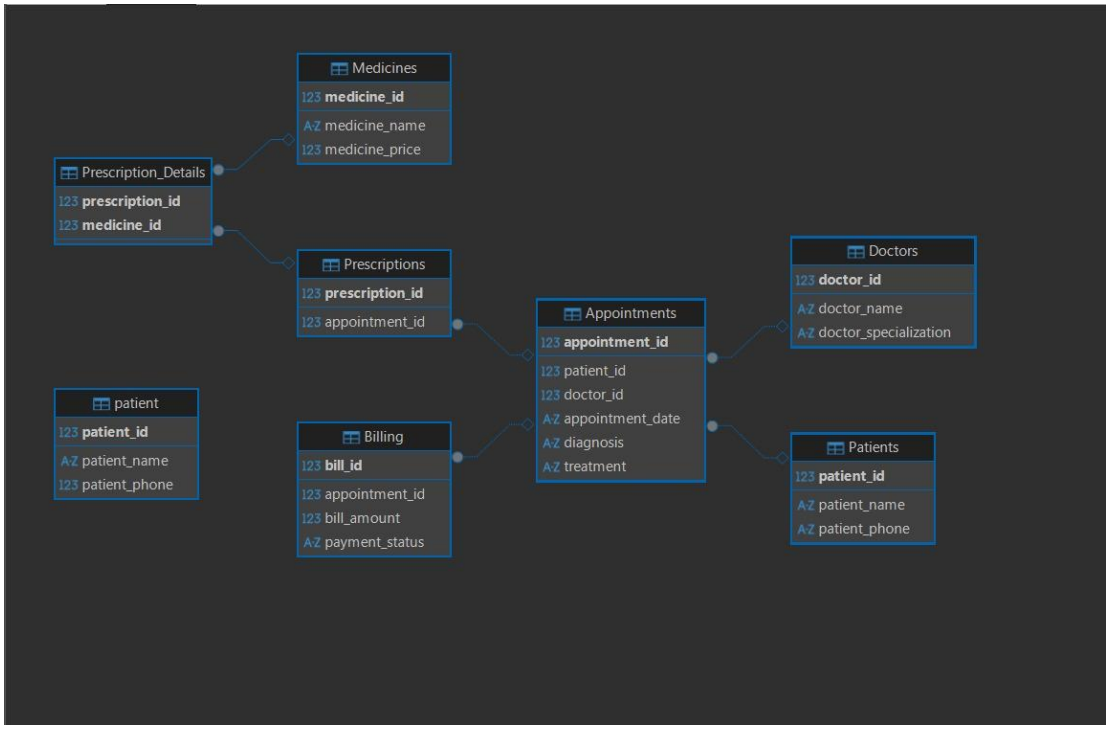
Note this two I will provide .sql file in that it will be there ...thank u 🙏

Update During Development
The hospital has introduced an operational change.
A single consultation can involve multiple doctors, and each doctor must be associated with their contribution to the consultation.
Update your design to accommodate this change.

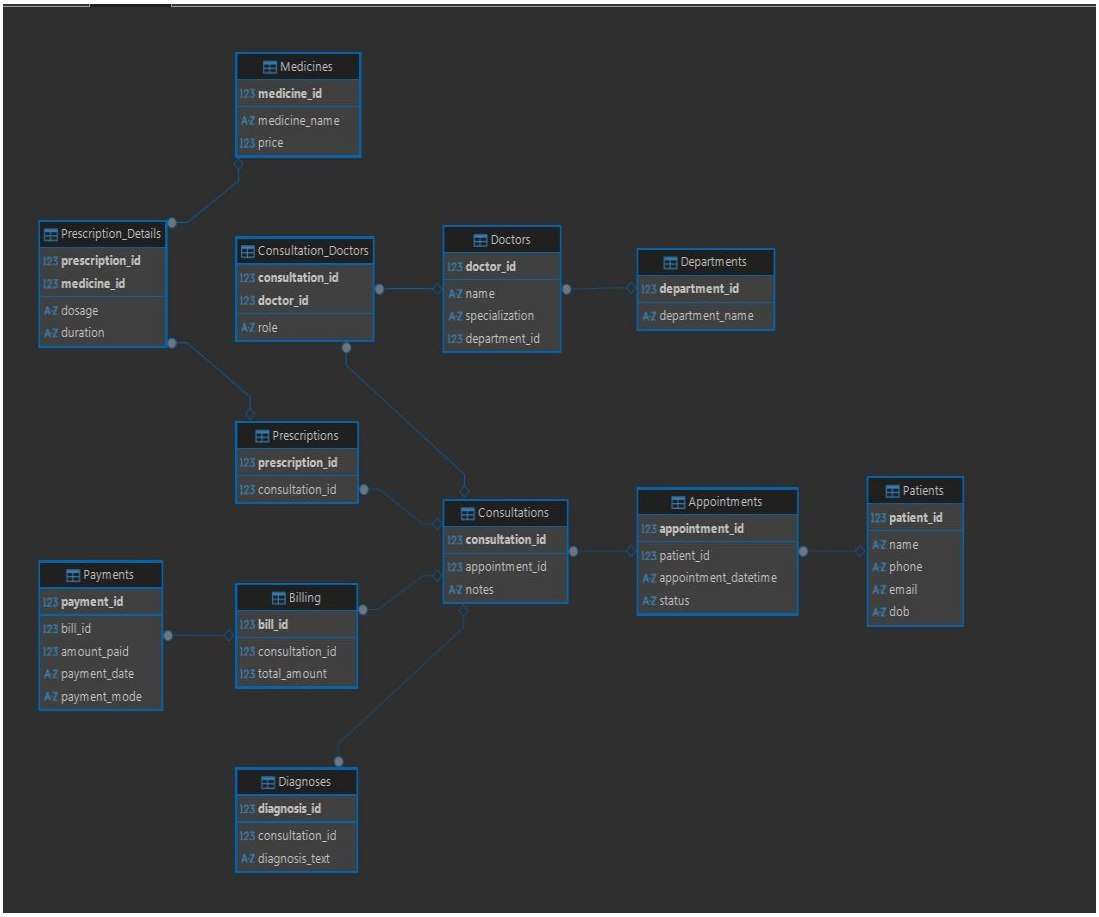
meaning is one consultation ->multiple consultation

```
Consultation_Doctors(
   consultation_id,
   doctor_id,
   role
    PRIMARY KEY (consultation_id, doctor_id),
   FOREIGN KEY (consultation_id) REFERENCES Consultations(consultation_id),
   FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id)
)
```

SECTION A



SECTION B



Section C

1.

```
  SELECT
  p.name AS patient_name,
  c.consultation_id,
  d.name AS doctor_name,
  dg.diagnosis_text,
  m.medicine_name
FROM Patients p
JOIN Appointments a ON p.patient_id = a.patient_id
JOIN Consultations c ON a.appointment_id = c.appointment_id
JOIN Consultation_Doctors cd ON c.consultation_id = cd.consultation_id
JOIN Doctors d ON cd.doctor_id = d.doctor_id
LEFT JOIN Diagnoses dg ON c.consultation_id = dg.consultation_id
LEFT JOIN Prescriptions pr ON c.consultation_id = pr.consultation_id
LEFT JOIN Prescription_Details pd ON pr.prescription_id = pd.prescription_id
LEFT JOIN Medicines m ON pd.medicine_id = m.medicine_id
WHERE p.patient_id = 1;
```

2.

```
SELECT
  d.doctor_id,
  d.name,
  COUNT(cd.consultation_id) AS total_consultations
FROM Doctors d
JOIN Consultation_Doctors cd ON d.doctor_id = cd.doctor_id
GROUP BY d.doctor_id, d.name
ORDER BY total_consultations DESC
LIMIT 1;
```

3.

```
SELECT
  p.patient_id,
  p.name,
  (b.total_amount - IFNULL(SUM(pay.amount_paid), 0)) AS outstanding_amount
FROM Patients p
JOIN Appointments a ON p.patient_id = a.patient_id
JOIN Consultations c ON a.appointment_id = c.appointment_id
JOIN Billing b ON c.consultation_id = b.consultation_id
LEFT JOIN Payments pay ON b.bill_id = pay.bill_id
```

```
GROUP BY p.patient_id, p.name, b.total_amount
HAVING outstanding_amount > 0;
```

4.

```
START TRANSACTION;

INSERT INTO Consultations (consultation_id, appointment_id, notes)
VALUES (3, 1, 'Follow-up visit');

INSERT INTO Diagnoses (diagnosis_id, consultation_id, diagnosis_text)
VALUES
(4, 3, 'Recovered'),
(5, 3, 'Routine Check');

INSERT INTO Prescriptions (prescription_id, consultation_id)
VALUES (3, 3);

INSERT INTO Prescription_Details (prescription_id, medicine_id, dosage,
duration)
VALUES
(3, 1, '1 tablet', '3 days'),
(3, 2, '1 tablet', '2 days');

INSERT INTO Billing (bill_id, consultation_id, total_amount)
VALUES (3, 3, 200.00);

INSERT INTO Payments (payment_id, bill_id, amount_paid, payment_date,
payment_mode)
VALUES (3, 3, 200.00, '2026-04-28', 'UPI');

COMMIT;
```